Name: Ronit Sahoo
Class: D20C
Roll No: 29
Batch: B

BLOCKCHAIN EXPERIMENT NO: 1

Aim - Write a Python program to understand SHA and Cryptography in Blockchain, and Merkle Root Tree Hash.

Tasks Performed:

1. **Hash Generation using SHA-256:**

   Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. **Target Hash Generation with Nonce:**

   Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. **Proof-of-Work Puzzle Solving:**

   Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. **Merkle Tree Construction:**

   Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program & Output:

1. Hash Generation using SHA-256

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                           #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:",hash_result )                                          #error statement
```

Output:

```
Enter a string: ronit
Hash: e688f6ce6c075e416ca1bbafb9f16c83db7ca52db1223cbb36bd94f5d2f9f7df
```

2. Target Hash Generation with Nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
```

```
# Concatenate the string and nonce
hash_string = input_string + nonce                                    # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                          # Error
```

Output:

```
Enter a string: ronit
Enter the nonce: sahoo
Hash Code: 007812ec03a6b3bb89563ebdd09d28eda2c1419e31f67acc2ad044741969daf6
```

3. Proof-of-Work Puzzle Solving

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Hello Blockchain By Ronit Sahoo"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output:

```
Hash: 0bed62a7eea6cd42630ba37cdb67a8fb3d5d6cf0ecbfe444775960b6cb56964b
Input String: Hello Blockchain By Ronit Sahoo
Leading Zeros: 1
Expected Nonce: 19
```

4. Merkle Tree Construction

```
import hashlib

def build_merkle_tree(transactions):                                    # Error
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(hash_combined)                      # Error

        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = ["Transaction 1", "Transaction 2", "Transaction 3", "Transaction 4",
"Transaction 5","Transaction 6", "Transaction 7"]

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)
```

Output:

```
Merkle Root: eee104fd82c121a28e359939713218466efd49ee4cdc8e60189fc5338b84f319
```

All the intermediate hash during the construction of the Merkle Tree Root Hash:

```
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    level = 0

    while len(transactions) > 1:
        print(f"\n--- Level {level} ---")
        print("Nodes:", transactions)

        # If odd number of nodes, duplicate the last node
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i + 1]
```

```
            hash_combined = hashlib.sha256(
                combined.encode("utf-8")
            ).hexdigest()

            print("Hash of:")
            print(f"  {transactions[i]}")
            print(f"  {transactions[i + 1]}")
            print(f"= {combined}")
            print(f"= {hash_combined}\n")

            new_transactions.append(hash_combined)

        transactions = new_transactions
        level += 1

    print("\n--- Merkle Root ---")
    print(transactions[0])

    return transactions[0]

transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $50",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Bob -> Bob : $50"
]

merkle_root = build_merkle_tree(transactions)

print("\nFinal Merkle Root:", merkle_root)
```

```
--- Level 0 ---
Nodes: ['Alice -> Bob : $200', 'Bob -> Dave : $50', 'Dave -> Eve : $100', 'Eve -> Alice : $300', 'Bob -> Bob : $50']
Hash of:
  Alice -> Bob : $200
  Bob -> Dave : $50
= Alice -> Bob : $200Bob -> Dave : $50
= 442768ae1db0fb344710e6ecf3156714405d1d2bb68a6860470d171d05909afc

Hash of:
  Dave -> Eve : $100
  Eve -> Alice : $300
= Dave -> Eve : $100Eve -> Alice : $300
= adaa076a7f644fa55a95170c6a0e383c952334dbcf824ae9b1c7bc00e85bcf7c

Hash of:
  Bob -> Bob : $50
  Bob -> Bob : $50
= Bob -> Bob : $50Bob -> Bob : $50
= 1119284f95279972a47c2d7ebe7fa6962c18c210744602b1f5e78825fab6f795
```

```
--- Level 1 ---
Nodes: ['442768ae1db0fb344710e6ecf3156714405d1d2bb68a6860470d171d05909afc', 'adaa076a7f644fa55a95170c6a0e383c952334dbcf824ae9b1c7bc00e85bcf7c', '1119284f95279972a
Hash of:
  442768ae1db0fb344710e6ecf3156714405d1d2bb68a6860470d171d05909afc
  adaa076a7f644fa55a95170c6a0e383c952334dbcf824ae9b1c7bc00e85bcf7c
= 442768ae1db0fb344710e6ecf3156714405d1d2bb68a6860470d171d05909afcadaa076a7f644fa55a95170c6a0e383c952334dbcf824ae9b1c7bc00e85bcf7c
= f6657c33b07779cf3d65b0d1f6f79f2174b85760576e8bfc8d290768a5babf2a

Hash of:
  1119284f95279972a47c2d7ebe7fa6962c18c210744602b1f5e78825fab6f795
  1119284f95279972a47c2d7ebe7fa6962c18c210744602b1f5e78825fab6f795
= 1119284f95279972a47c2d7ebe7fa6962c18c210744602b1f5e78825fab6f7951119284f95279972a47c2d7ebe7fa6962c18c210744602b1f5e78825fab6f795
= 8a526a5aa32d0f022ceb8ff91d78c34a201f384fcba5fe0a2c739c5c7cf8c4f9


--- Level 2 ---
Nodes: ['f6657c33b07779cf3d65b0d1f6f79f2174b85760576e8bfc8d290768a5babf2a', '8a526a5aa32d0f022ceb8ff91d78c34a201f384fcba5fe0a2c739c5c7cf8c4f9']
Hash of:
  f6657c33b07779cf3d65b0d1f6f79f2174b85760576e8bfc8d290768a5babf2a
  8a526a5aa32d0f022ceb8ff91d78c34a201f384fcba5fe0a2c739c5c7cf8c4f9
= f6657c33b07779cf3d65b0d1f6f79f2174b85760576e8bfc8d290768a5babf2a8a526a5aa32d0f022ceb8ff91d78c34a201f384fcba5fe0a2c739c5c7cf8c4f9
= b84732d6057b1d5579a347dc63e0cdc0fe089ebe521302fad092b21658be506f


--- Merkle Root ---
b84732d6057b1d5579a347dc63e0cdc0fe089ebe521302fad092b21658be506f

Final Merkle Root: b84732d6057b1d5579a347dc63e0cdc0fe089ebe521302fad092b21658be506f
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

1. SHA-256 Hashing provides a secure and unique fingerprint for any input, ensuring data integrity.
2. Nonce and Target Hashing allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
3. Proof-of-Work Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
4. Merkle Tree Construction efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.